

Modulare C-Projekte mit C Make

Für kleine Projekte, die nur aus einer *main.c* bestehen, reicht ein einfacher Befehl wie:

```
gcc main.c -o programm
```

Aber sobald dein Projekt aus mehreren *.c*- und *.h*-Dateien besteht, wird das Kompilieren schnell unübersichtlich und nervig.

Genau hier hilft dir **CMake**:

Es sorgt dafür, dass dein Projekt **sauber strukturiert, plattformübergreifend kompiliert** (Windows, Linux, Mac), **Bibliotheken** wie SDL leicht eingebunden werden können und du das Ganze mit nur **wenigen Befehlen automatisch bauen** kannst.

Was macht CMake?

CMake ist kein Compiler, sondern ein Werkzeug, das dir hilft, dein Projekt automatisch für verschiedene Systeme baubar zu machen.

Statt jeden *gcc*-Befehl von Hand zu schreiben, erstellst du einfach eine Datei namens *CMakeLists.txt*. Darin steht, wie dein Projekt aufgebaut ist, welche Dateien dazugehören, welche Bibliotheken du brauchst, usw.

CMake liest diese Datei und erstellt daraus die passenden **Build-Dateien für dein System**:

- Auf **Linux** z. B. ein *Makefile*
- Auf **Windows** eine Visual-Studio-Projektdatei oder Makefiles für MinGW
- Auf **Mac** optional ein Xcode-Projekt

Du musst dann nur noch *cmake* + *make* (oder dein Build-Tool) ausführen – und der ganze Bauvorgang läuft automatisch.

So sparst du dir händisches Kompilieren, und dein Projekt funktioniert **auf allen Plattformen** gleich.

CMake installieren

Windows

1. [CMake von cmake.org downloaden](https://cmake.org/download)
2. Bei der Installation „**Add CMake to system PATH**“ aktivieren
3. Optional: MinGW oder Visual Studio installieren
4. Terminal öffnen und testen:

```
cmake --version
```

Linux

Die meisten Linux-Distributionen bringen CMake schon mit oder haben es in ihren Paketquellen. Installieren kannst du es einfach mit deinem Paketmanager, z. B.

```
sudo apt install cmake      # Debian, Ubuntu
sudo dnf install cmake      # Fedora
sudo pacman -S cmake        # Arch
```

ob es geklappt hat prüfst du gleich wie bei Windows

Projektstruktur beispiel

```
MeinProjekt/  
├─ src/  
│   ├── main.c  
│   ├── headers/  
│   │   ├── main.h  
│   │   └─ irgendwas.h  
│   └─ weiteres/  
│       └─ irgendwas.c  
├─ CMakeLists.txt  
└─ build/
```

Ordner *src/* für Quellcode, *headers/* für Header, *build/* für alles was CMake erzeugt und *weiteres/* für die erweiterung unseres codes

CmakeLists.txt im Überblick

```
cmake_minimum_required(VERSION 3.10)  
project(MeinProjekt C)  
  
# Alle Source-Dateien  
set(SOURCES  
    src/main.c  
    src/weiteres/irgendwas.c  
)  
  
# Include-Verzeichnisse (damit #include "main.h" gefunden wird)  
include_directories(src/headers)  
  
# Erstelle die ausführbare Datei  
add_executable(mein_programm ${SOURCES})
```

Erklärung

Zeile	Bedeutung
<code>cmake_minimum_required(VERSION 3.10)</code>	Gibt an, welche CMake-Version mindestens nötig ist
<code>project(MeinProjekt C)</code>	Setzt den Projektnamen und die Sprache (C)
<code>set(SOURCES ...)</code>	Listet alle .c-Dateien auf, die kompiliert werden sollen
<code>include_directories(src/headers)</code>	Damit dein Code <code>#include "main.h"</code> findet
<code>add_executable(...)</code>	Baut aus den .c-Dateien eine ausführbare Datei

Kompilieren (Crossplattform)

Im root directory deines projektes gibst du folgendes ein:

```
cmake -B build/  
cmake --build build/
```

Danach findest du die fertige Datei *mein_programm* im *build/* Ordner

Bibliotheken einbinden

```
set(SDL2_DIR "C:/SDL2")  
include_directories(${SDL2_DIR}/include)  
link_directories(${SDL2_DIR}/lib)  
  
target_link_libraries(mein_programm SDL2 SDL2main)
```

Das musst du machen, **wenn du z. B. SDL2 nutzt**. Pfad musst du an dein System anpassen.

In Kurzform

- CMake ist Pflicht, wenn du mit mehreren Dateien arbeitest
- Plattformübergreifend, sauber und einfach erweiterbar
- Macht dein Leben **sehr viel leichter**